
Informe de Implementacion Sistema de Inteligencia Artificial

FONOKIT - Plataforma de Fonoaudiologia

Fecha: 01 de April de 2026 | Version 1.0

7
Edge Functions

6
Modelos HF

3
Tablas Nuevas

\$0
Costo Modelos

Contenido

1. Resumen Ejecutivo
2. Arquitectura del Sistema
3. Modelos de IA Implementados
4. Edge Functions Deployeadas
5. Infraestructura de Base de Datos
6. Admin Dashboard (antes vs despues)
7. Auditoria Tecnica Realizada
8. Meta Pixel y Tracking
9. Deploy Automatico (CI/CD)
10. Proximos Pasos

1. Resumen Ejecutivo

Se implemento un sistema completo de inteligencia artificial para la plataforma Fonokit, utilizando modelos gratuitos de HuggingFace como backbone principal. El sistema incluye 7 Edge Functions desplegadas en Supabase que cubren: sugerencias de tratamiento, interpretacion de evaluaciones clinicas (ADI-R, ADOS-2, Perfil Sensorial), generacion de material terapeutico, analisis de progreso del paciente, y recomendaciones de productos del marketplace.

Ademas se realizo una auditoria tecnica completa del repositorio que incluyo: versionamiento del schema de base de datos (186 tablas), habilitacion de Row Level Security en las 173 tablas activas, consolidacion de 13 tablas duplicadas, centralizacion del logging (0 console.log en produccion), estandarizacion del error handling, y optimizacion de queries N+1.

173
Tablas con RLS

431
Policies

271
Archivos con Logger

99
Funciones apiHandler

2. Arquitectura del Sistema

La arquitectura se basa en Edge Functions de Supabase (Deno/TypeScript) que actuan como gateway entre el frontend React y los modelos de HuggingFace. Un cliente compartido (_shared/hf-client.ts) centraliza las llamadas a la API de HF con retry automatico para cold starts y rate limits.

Capa	Tecnologia	Proposito
Frontend	React + Vite	UI del admin y terapeutas
API Gateway	Supabase Edge Functions	7 funciones Deno/TypeScript
AI Models	HuggingFace Inference API	Mistral-7B, BART, MiniLM, Whisper
Chat AI	Anthropic Claude Haiku	Chatbot de pacientes
Database	Supabase PostgreSQL + pgvector	173 tablas + embeddings vectoriales
Shared Client	_shared/hf-client.ts	Retry, fallback, PII stripping

3. Modelos de IA Implementados

Modelo	Tipo	Uso en Fonokit	Costo
Mistral-7B-Instruct-v0.3	Generacion	Tratamientos, evaluaciones, material	Gratis
BART-Large-CNN	Resumen	Resumir transcripciones clinicas	Gratis
BART-Large-MNLI	Clasificacion	Clasificar tendencias de progreso	Gratis
MiniLM-L6-v2	Embeddings	Busqueda semantica marketplace	Gratis
Whisper-Small	Transcripcion	Audio a texto en sesiones	Gratis
Claude Haiku 4.5	Chat	Chatbot pacientes/terapeutas	~\$0.001/ call

Estrategia de Rate Limits (HF Free Tier)

- Header x-wait-for-model: true en todas las requests (espera cold start)
- Retry automatico: 3 intentos con backoff 10s / 20s / 30s
- Quotas por usuario: 20 calls/dia free, 100 premium (tabla ai_usage_quotas)
- Cola async (ai_task_queue) para operaciones bulk no urgentes
- Fallback chain: cada feature tiene degradacion graceful

4. Edge Functions Deployadas

Edge Function	Modelos	Input	Output
suggest-treatment	Mistral-7B	Diagnostico + historial	Plan de tratamiento JSON
evaluate-analysis	Mistral-7B	Scores ADI-R/ADOS-2/Sensorial	Interpretacion clinica
generate-material	Mistral-7B	Diagnostico + tipo + edad	Ejercicios terapeuticos
analyze-progress	BART-MNLI + Mistral	Actividades de sesion	Reporte de tendencias
recommend-purchases	MiniLM embeddings	Contexto paciente	Items marketplace
process-notiz	Whisper + Claude	Audio de sesion	Notas clinicas
new-meta-capi	N/A	Eventos browser	Tracking server-side

Prompt Templates Editables

Se crearon 4 prompt templates almacenados en la tabla ai_prompt_templates que son editables desde el admin dashboard. Cada template usa variables con formato {{variable}} que se reemplazan dinamicamente con datos del paciente. Los templates cubren: sugerencias de tratamiento, interpretacion de evaluaciones, generacion de material, y analisis de progreso.

5. Infraestructura de Base de Datos

Nuevas Tablas para AI

Tabla	Proposito	Columnas Clave
ai_task_queue	Cola async para jobs AI	task_type, input/output_data, status
ai_embeddings	Vectores para busqueda semantica	source_table, embedding vector(384)
ai_prompt_templates	Prompts editables desde admin	template, variables, model, category

Extension PostgreSQL

Se habilito pgvector para almacenar embeddings de 384 dimensiones (MiniLM-L6-v2) que permiten busqueda semantica por similitud coseno contra el catalogo del marketplace.

Datos Disponibles para Entrenamiento

Dataset	Tabla	Tipo de Dato
Sesiones Notiz	notiz_sessions	Transcripciones + analisis
Historia Clinica	clinical_history	Notas de sesion estructuradas
Planes de Tratamiento	treatment_plans	Objetivos + actividades JSON
Evaluaciones ADI-R	adir_evaluations + items	Scores por dominio A/B/C/D
Evaluaciones ADOS-2	ados2_evaluations + items	Scores AS/CRR/COM
Perfil Sensorial	sensorial_evaluations + items	Scores por seccion sensorial
Actividades de Sesion	session_activities	Logros: logrado/en_proceso/no_logrado
Feedback AI	ai_feedback	Aceptado/rechazado/modificado

6. Admin Dashboard: Antes vs Despues

Componente	Antes	Despues
fetchModels()	4 modelos hardcodeados	6 modelos reales con provider/model_id
fetchUsageStats()	Conteo basico	Notiz + templates + chat + quotas + tokens
runInference()	return "Mock response"	supabase.functions.invoke() real
fetchTrainingJobs()	return []	Query a ai_task_queue
Prompt Templates	return { success: false }	CRUD real contra ai_prompt_templates
Datasets	Solo conteos	Conteos de 8 fuentes de datos
Settings	return { success: true }	CRUD real contra ai_settings
Evaluaciones	return []	Stats de ai_feedback (accepted/rejected)

7. Auditoria Tecnica Realizada

Item	Antes	Despues	Impacto
Schema versionado	2 archivos SQL	7 migraciones + baseline	Infraestructura
Tablas duplicadas	186 tablas	173 tablas (-13)	Limpieza
RLS (seguridad)	20 tablas sin RLS	173/173 con 431 policias	Seguridad
Console.log	~1,192 en produccion	0 (logger centralizado)	Produccion
Error handling	Inconsistente	99 funciones con apiHandler	Estabilidad
Tipos TypeScript	0 lineas	15,356 lineas generadas	Type safety
Query N+1	8 roundtrips/ficha	1 roundtrip (RPC)	Performance
Permisos	4 contextos duplicados	1 sistema unificado	Mantenibilidad

8. Meta Pixel y Conversions API

Se configuro el Meta Pixel (ID: 1464610018368997) con doble tracking: browser-side via fbq() y server-side via la edge function new-meta-capi que envia eventos a la Meta Conversions API. Ambas vias comparten un event_id UUID para deduplicacion.

Configuracion	Valor
Pixel/Dataset ID	1464610018368997
Business ID	512800222455177
System User	Conversions API System User (61586790467616)
Token expiration	Nunca
Supabase Secret	META_CAPI_ACCESS_TOKEN configurado
Edge Function	new-meta-capi desplegada

9. Deploy Automatico (CI/CD)

Se configuro GitHub Actions para deploy automatico a Hostinger via FTP. Cada push a main ejecuta: npm ci, npm run build, y sube dist/ completo a public_html con dangerous-clean-slate que elimina assets viejos antes de subir. Esto resuelve el problema de pagina en blanco que ocurría con el deploy manual.

Paso	Herramienta	Detalle
1. Checkout	actions/checkout@v4	Descarga codigo del repo
2. Setup Node	actions/setup-node@v4	Node 20 + npm cache
3. Install	npm ci	Instala dependencias
4. Build	npm run build	Vite build con env vars de secrets
5. Deploy	FTP-Deploy-Action@v4.3.5	Sube dist/ a Hostinger, limpia viejos

10. Proximos Pasos

Fase	Descripcion	Esfuerzo	Impacto
Fase 2	UI de sugerencias en ficha paciente	Medio	Alto
Fase 3	Embeddings batch + busqueda semantica	Medio	Medio-Alto
Fase 4	FonoLevel AI scoring + pipeline training data	Alto	Medio
Futuro	Fine-tuning con HF AutoTrain (\$9/mo)	Alto	Alto

Sobre Fine-Tuning

El fine-tuning real de modelos en HuggingFace requiere HF Pro (\$9/mes) o GPU local. Lo implementado actualmente usa few-shot prompting via ai_prompt_templates como forma de "entrenamiento" inmediato. Se crearon pipelines de datos (ai_task_queue) y estructura de feedback (ai_feedback, ai_recommendation_feedback) que permitiran fine-tuning futuro cuando haya presupuesto. Los datos se exportan en formato JSONL listo para AutoTrain.